

Experiment 16 — Spectral PDE: an FFT inside a time loop

Mathilda — execution-speed experiments

Contents

Experiment 16 — Spectral PDE: an FFT inside a time loop	1
Hypothesis	1
The kernels	1
Results — nothing needed fixing	2
Where the remaining 1.3–1.9× is	2
What is still open	2
Why Mathilda is not the fastest here, and what it would take	2
Verification	3

Experiment 16 — Spectral PDE: an FFT inside a time loop

Date: 2026-07-31 · Code: no change required · Result: 2D Navier-Stokes 18.4× faster than Mathematica; the whole group within 1.3–1.9× of NumPy, with no fix needed

Common method in [README.md](#).

Hypothesis

The suite has two spectral rows and both time one large transform: Fourier of 2^{20} reals and of 2^{20} complexes. FFTW's inner kernels dominate them completely, so what they measure is FFTW.

A pseudo-spectral PDE solver has the opposite profile. It runs thousands of medium transforms with complex elementwise algebra between them, so what it measures is per-call overhead, complex-array arithmetic, and whether an Fourier result stays on the buffer for the next operation. Nothing in the suite had run that, and complex packed arrays had never been exercised at all outside a single transform.

The kernels

id	kernel	what it stresses
ks	Kuramoto-Sivashinsky, 2048 modes, 2000 steps	4000 rank-1 transforms, complex elementwise
ns2d	2D Navier-Stokes vorticity, 128^2 , 200 steps	1000 rank-2 transforms, 2/3 dealiasing
poissonfft	FFT Poisson solve, 512^2 , 30 solves	60 large rank-2 transforms

The convention, which is load-bearing

Wolfram's Fourier is $(1/\sqrt{n}) \sum x e^{+2\pi i \dots}$ — a positive exponent — which is NumPy's `ifft(norm="ortho")`, and InverseFourier is `fft(norm="ortho")`. Getting that backwards produces a plausible-looking wrong answer: the solution still evolves, still looks like turbulence, and is a different equation. The cross-system value

check is what stops that, and it is the reason the check is worth more here than anywhere else in the suite.

The checks are deliberately short runs — 50 KS steps, 20 NS steps. Both equations are chaotic, so the state at the final time is a property of the arithmetic rather than of the equation, and asking three independently written solvers to agree on it to six figures would be meaningless.

Results — nothing needed fixing

Benchmark	Mathilda	Mathematica 14.0	NumPy 2.4.4	vs WL	vs NumPy
Kuramoto-Sivashinsky, 2048 modes, 2000 steps	332 ms	250 ms	173 ms	1/1.33×	1/1.92×
2D Navier-Stokes, 128 ² , 200 steps	551 ms	10.13 s	246 ms	18.37×	1/2.24×
FFT Poisson, 512 ² , 30 solves	312 ms	187 ms	199 ms	1/1.67×	1/1.56×

This is the second group in the sweep that needed no code change, and the reason is that the pieces were already there: `Fourier` and `InverseFourier` were made packed-aware in experiment 6 and reach FFTW directly, complex arithmetic on a `NDT_COMPLEX64` buffer is a threaded kernel, and `Nest` — which is how a time loop is written — was put on the aware list in experiment 10 after it was found materialising its state once per iteration.

Measured directly: 2²⁰ complex `Fourier` is 41 ms, complex add on 2²⁰ is 2.2 ms, complex multiply 8.7 ms, Abs 5.8 ms. The KS row runs 4000 transforms of 2048 plus about ten elementwise passes per step in 332 ms — 166 μ s per step.

The Navier-Stokes row is the interesting one. Mathilda is 18.4× faster than Mathematica on it, which is the largest margin in the whole five-sweep suite, and the shape is the same as the Verlet row in experiment 13 and the ray tracer in experiment 19: a loop whose body is a handful of whole-array operations. Mathematica appears to pay a per-operation cost there that its own single-large- transform rows do not show.

Where the remaining 1.3–1.9× is

Not in the FFT. Per step, the KS row does two transforms of 2048 (~30 μ s together, FFTW) and roughly ten elementwise passes over 2048 complexes (~4 μ s of memory traffic). The rest — about 130 μ s — is evaluation overhead: ten `evaluate_step` calls, ten packed-argument gate checks, ten result allocations, per time step.

That is the same ceiling experiments 15 and 18 run into, and it is the honest description of where Mathilda now sits: the array kernels are at or near the machine, and what separates it from NumPy on short-body loops is the cost of getting to them. NumPy pays that cost too — a NumPy expression is also a sequence of C calls from a bytecode interpreter — but its per-call cost is lower because it does not have to decide anything about heads, attributes or exactness on the way in.

What is still open

- Per-call dispatch on a short loop body is now the dominant term in three separate experiments (15, 16, 18). It is not a missing fast path and no further packing work will move it; it is plan item 5.2, and it wants its own experiment with a profile rather than a hypothesis.
- **poissonfft** is 1.56× NumPy on 60 transforms of 512², which is nearly all FFTW in both systems. The difference is the two elementwise passes between the transforms and the allocation of a 2 MB complex result per transform.

Why Mathilda is not the fastest here, and what it would take

row	Mathilda	best other	gap	cause
Kuramoto-Sivashinsky	332 ms	173 ms (NumPy)	1.92×	per-call cost on 2048-element complex passes

row	Mathilda	best other	gap	cause
2D Navier-Stokes	551 ms	246 ms (NumPy)	2.24×	the same, six transforms deep
FFT Poisson	312 ms	187 ms (WL)	1.67×	two elementwise passes and a 2 MB result per transform

Splitting one Kuramoto-Sivashinsky step across all three systems — the `.m` and `.py` files print this — locates the gap exactly:

per step	Mathilda	Mathematica	NumPy
InverseFourier (one transform)	32 μ s	32 μ s	37 μ s
Fourier[u u] (transform + one real product)	55 μ s	36 μ s	41 μ s
one complex elementwise pass	12 μ s	7 μ s	4 μ s
the whole step	159 μ s	147 μ s	92 μ s

The transforms are at parity — FFTW in Mathilda, and NumPy's inverse is actually marginally slower. The entire gap is the middle row: one elementwise pass over 2048 complex numbers (32 KB) costs 12 μ s against NumPy's 4 μ s, and a step makes six or seven of them.

32 KB at 12 μ s is 2.7 GB/s, far below this machine's bandwidth, so that 12 μ s is not memory — it is the fixed cost of an array operation, paid on an array small enough that the fixed cost dominates.

The road to fastest

1. Cut the per-operation constant (plan 5.2). Six passes $\times$ ~ 8 μ s of overhead is ~ 50 μ s of a 159 μ s step, and the whole gap to NumPy is 67 μ s. This single item is the row. It is the same item as experiments 15 and 18, and this experiment is the cleanest place to measure it because the surrounding work (FFTW) is already known to be at parity.
2. Fuse the coefficient-space algebra. `(uh + dt nl) ksdn` is three passes and two temporaries; `(0.5 I kskk) Fourier[u u]` is two more. A fused complex elementwise evaluator would make the non-transform half of the step one pass, which is worth more than removing the overhead of each.
3. Reuse the transform's output buffer. Each `Fourier` allocates a fresh 2048-complex (32 KB) result, six times per step, 2000 steps. An output-buffer cache keyed on shape and dtype — the same idea as an FFTW plan cache, which already exists — removes 12000 allocations per run.

There is nothing to fix in the spectral machinery itself: this experiment needed no code change and Mathilda is 18.4× ahead of Mathematica on the Navier-Stokes row. Being fastest is a matter of the evaluator's constant, not of the transform.

Verification

- No source change was made for this experiment.
- All three values agree across all three systems, which given the transform convention is the substantive check: an inverted sign or a missing $1/\sqrt{n}$ changes the answer without changing its plausibility.